

Unidad 1

Cómo Funciona?

Si un proceso está ejecutando un programa en modo usuario y necesita un servicio del sistema, como por ejemplo leer datos de un archivo, debe ejecutar una instrucción **trap** para transferir el control del procesador al sistema operativo. El sistema operativo debe detectar cual es el proceso llamador por medio de sus parámetros. Entonces busca la llamada al sistema y devuelve el control a la siguiente instrucción de la llamada al sistema. Podemos decir que una llamada al sistema es como una llamada a procedimiento solo que esta no puede acceder al kernel del sistema operativo y la llamada la sistema sí. Para ejemplificar una llamada al sistema vamos a utilizar el system call **read**. Como casi todas las llamadas al sistema, esta es invocada desde un programa C llamando a un procedimiento de una librería el cual tiene el mismo nombre que la system call **read**.

```
count = read(fd, buffer, nbytes);
```

- **fd**: File descriptor es la ruta en donde se encuentra el archivo que se quiere leer.
- **buffer**: Es un puntero en donde se van a guardar los datos de dicho archivo.
- **nbytes**: Es la cantidad máxima de bytes que se permite leer.

Las system call se realizan a traves de una serie de pasos. Utilizaremos **read** como ejemplo:

1. El programa llamador ubica los parametros de manera inversa **de read o de su función main?** dentro del **stack**. Como se muestra en el paso 1-3 en la figura 1-17. El primer y tercer parametro son pasador por valor y el segundo es llamado por referencia ya que es un puntero.
2. Luego se ejecuta la llamada a todos los procedimientos (paso 4)
3. El procedimiento de la librería probablemente está escrito en assembly por lo tanto coloca el número de la system call en un registro conocido por el sistema operativo (paso 5)
4. Ejecuta la instrucción **trap** para cambio de modo usuario a modo kernel y comienza la ejecución en una dirección fija dentro del kernel (paso 6)
5. La instrucción **trap** es parecida en cierto sentido a una ejecución de una llamada a rutina en el sentido que la siguiente instrucción a ejecutar se toma de una dirección de memoria lejana y la dirección de retorno es guardada en el stack para utilizarse después. Sin embargo the instrucción **TRAP** se diferencia una llamada a procedimiento de dos formas. Primero tiene un efecto colateral que es que cambia a modo kernel. Segundo, en lugar de ir a una dirección de memoria específica donde el procedimiento está ubicado, la instrucción **TRAP** no puede saltar directamente a una dirección de memoria arbitraria. Dependiendo de la arquitectura solo puede saltar a una dirección fija o hay un campo de 8 bits en la instrucción que nos da el índice a una tabla ubicada en memoria la cual contiene las direcciones a las que puede saltar. Luego el código del kernel que le sigue al **trap** examina el número de la system call y despacha el manejador de system calls correspondiente usualmente utilizando la tabla de punteros la cual apunta los distintos manejadores de system calls (paso 7).
6. En este punto comienza a ejecutarse el manejador de system calls (paso 8)
7. Una vez terminado el trabajo, se devuelve el control al espacio librerías de usuario indicado por la instrucción **TRAP** (paso 9)
8. Este procedimiento devuelve un resultado al programa de usuario como toda llamada a procedimientos (paso 10)

Procesos

El kernel tiene la capacidad de ejecutar programas almacenados en el sistema. Cuando un programa se encuentra en ejecución lo llamamos **proceso**. El sistema operativo controla la **creación**, **ejecución** y **finalización** de dicho proceso.

Un proceso se crea cuando:

- En la secuencia de **inicio del sistema**.
- Cuando una aplicación realiza un **system call** para crear un proceso.
- Cuando un usuario solicita ejecutar un programa ya sea traves de la linea de comandos o haciendo doble click sobre un ejecutable.

El estado de un proceso en ejecución es representado por el valor que tiene el registro **program counter** o **PC** y el contenido de los registros del procesador. El diseño de la memoria asignada a un proceso generalmente se divide in multiple secciones. Estas secciones son:

- Sección de texto: Donde se encuentra el código ejecutable.
- Sección de datos: Variables globales.
- Sección Heap: Memoria que es asignada dinámicamente durante el tiempo de ejecución de un programa.
- Sección Stack: Almacen temporal de datos cuando se invocan funciones se almacenan elementos como parametros, direcciones de retorno y variables locales

Podemos ver que las secciones de texto y datos tienen un tamaño fijo es decir no cambiar durante el tiempo de ejecución de un programa. Sin embargo las secciones heap y stack pueden crecer o decrecer dinámicamente durante el tiempo de ejecución de un programa.

Cada vez que se llama a una función un registro de activación contiene los parámetros de la función, variables locales y direcciones de retorno los cuales son ubicados dentro de la pila. Cuando el control es devuelto de la función el registro de activación es quitado de la pila.

Aunque dos procesos esten asociados al mismo programa ellos sin embargo son considerados dos secuencias de ejecución separadas. Cada uno tendra sus secciones texto, datos, heap, stack

Creación de Procesos

Durante el curso de la ejecución de un proceso dicho proceso puede crear varios procesos nuevos. El proceso creador es llamada **proceso padre** y los nuevos procesos son llamados procesos hijos. Cada uno de estos procesos hijos pueden crear nuevos procesos formando así un arbol de procesos.

La mayoría de los sistemas operativos (incluyendo Unix, Linux, Windows) identifican sus procesos de acuerdo a un identificador único llamado process identi o pid el cual es usualmente un número entero. Este identificador puede utilizarse como índice para acceder a varios atributos de un procesos dentro del kernel.

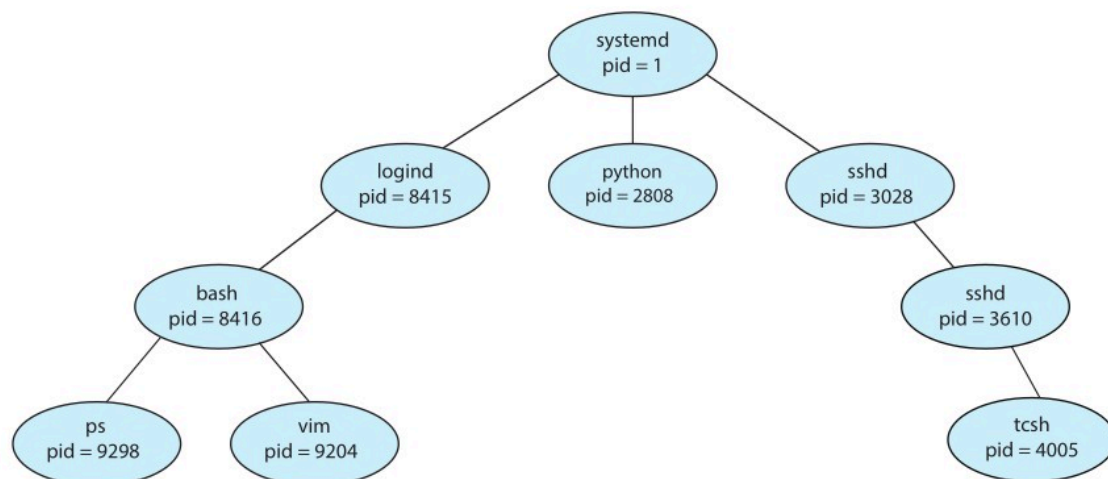


Figure 3.7 A tree of processes on a typical Linux system.

Cuando un proceso crea un nuevo proceso existen dos posibilidades con respecto a la ejecución:

1. El padre continua ejecutandose concurrentemente con su hijo.
2. El padre espera hasta que algun o todos sus hijos hallan terminado.

Estado de un Proceso

A medida que un proceso se ejecuta cambia de estado. El estado de un proceso se define en parte por la actividad actual de ese proceso. Un proceso puede estar en uno de los siguientes estados:

- **New:** El proceso ha sido creado.
- **Running:** Las instrucciones están siendo ejecutadas.
- **Waiting:** El proceso está en espera debido a la ocurrencia de un evento (Se completo un E/S o se recibio una llamada del sistema).
- **Ready:** El proceso está esperando a ser asignado a algún procesador.

Bloque de Control de Proceso (Process Control Block)

Cada proceso es representado en el sistema operativo mediante un PCB también llamada bloque de control de tarea. Este contiene distintas secciones con información asociada a ese proceso en especifico. Algunas de ellas son:

- **Process state:** Contiene información acerca del estado del proceso.
- **Program counter:** El contador de programa indica la dirección de memoria de la próxima instrucción que debe ejecutar el procesador.
- **Cpu register:** Los registros varían en cantidad y en tipo dependiendo de la aquitectura de la computadora. Entre ellos se incluyen el acumulador, puntero de pila, registros de índice y registros de proposito general. Toda esta información junto con el contador de programa debe ser resguardad cada vez que ocurre una interrupción esto es para permitir que le proceso siga ejecutandose una vez que se reasignado (rescheduled) a un procesador para que sea ejecutado.

- **Información de la planificación del cpu:** Contiene información sobre la prioridad del proceso, punteros a la cola de planificación y otros parametros de planificación.
- **Información de Administración de Memoria:** Contiene información de los registros limite y base, información sobre la tabla de páginas o la tabla de segmentos dependiendo del sistema de memoria utilizado en el sistema operativo.
- **Información de Conteo:** Contiene información sobre la cantidad de cpu utilizado en tiempo real, limites de tiempo, número de cuentas, trabajos o procesos.
- **Información del estado de E/S:** Esta información incluye la list de los dispositivos de entrada/salida asignados al proceso así como la lista de los archivos abiertos, etc.

En resumen el PCB sirve como repositorio de todos los datos necesarios para iniciar o reanudar un proceso además de los datos de conteo.

Estados de un proceso

Cuando se crea un proceso esta pasas por distintos estados durante su ciclo de vida algunos son nuevo, listo, ejecutando bloqueado, listo suspendido, bloqueado suspendido